REFERENCE HANDOUT

How language models work

The definitions, relationships and decision criteria from the workshop, for reference at work. Numbers in the right margin point to the section of the course *Tokens to Decisions* that covers the entry in full.

Full course `../course/index.html` **Workshop deck** `../presentation/index.html`
Getting started `../practical-guide/index.html`

Part One · How a language model works

LEARNING AND TRAINING

Machine learning 1.1 — rules are found by a search procedure rather than written by a person. In **supervised** learning you supply inputs paired with desired outputs. In **self-supervised** learning the targets are constructed from the data itself, which is how language models are trained: hide what comes next and predict it. No annotation is needed, so the supply of training data is effectively unlimited.

Neural network 1.2 — layers of simple units. Each unit computes a weighted sum of its inputs, adds a constant, and applies a non-linear function. Without that non-linearity, stacked layers collapse algebraically into a single layer, and a single layer cannot represent a problem as simple as exclusive-or.

Parameters 1.2 — the learned numbers: the **weights** multiplying each input and the **bias** added to the sum. A frontier model has on the order of 10^{11} of them. Distinct from **hyperparameters** — layer count, learning rate, batch size — which are chosen before training rather than learned.

> $z = w_1x_1 + w_2x_2 + b$
>
> $\hat{y} = \sigma(z) = 1 / (1 + e^{-z})$
>
> One unit with two inputs. A neural network consists of very many such units, arranged in layers.

Loss 1.3, 1.9 — one number saying how wrong a prediction was, and the complete specification of the training goal: whatever reduces it is what the model learns to do. Squared error, $\frac{1}{2}(\hat{y} - y)^2$, suits simple regression. Language models use **cross-entropy**, −log of the probability assigned to the token that actually occurred, which penalises assigning near-zero probability to something that then happens.

Gradient descent 1.4 — compute the slope of the loss with respect to every parameter, then step each one a small distance downhill. Too small a step and training crawls; too large and each step overshoots. In practice the gradient is estimated from a small random batch rather than the full dataset.

> $w \leftarrow w - \eta \cdot \partial L/\partial w$
>
> η is the learning rate. Training a large language model applies this rule repeatedly to around 10^{11} parameters.

Backpropagation 1.6 — an efficient method for obtaining every one of those derivatives in a single backward sweep, at roughly the cost of one extra forward pass. It computes derivatives; gradient descent does the learning. The two are routinely conflated and are not the same thing.

Generalisation and overfitting 1.7 — the goal is behaviour on inputs never seen. A model with enough capacity can memorise its training data and perform poorly on anything new; a held-out set is what detects this. Large models also retain verbatim fragments of their training data, which is a licensing question independent of quality.

Inference 1.8 — running the trained model forward. Parameters are frozen, nothing is written back, and nothing you do while using a model changes it. Apparent memory is always text that surrounding software re-sent.

FROM TEXT TO PREDICTION

Token 2.1 — a chunk of text drawn from a fixed vocabulary of roughly 100,000–200,000 entries, built before training by repeatedly merging the most frequent adjacent pair (**byte-pair encoding**). Frequency determines granularity: common words are one token, rare ones several. A leading space belongs to its token, and the model cannot see the characters inside one — which is why character-level tasks are oddly hard.

Language effects 2.5 — the vocabulary was built from a corpus dominated by English and code, so the same meaning costs more tokens in most other languages. That affects cost directly and, through the context window, how much material fits. Measure your own languages rather than assuming a ratio.

Embedding 2.3 — the first layer is a table with one row per vocabulary entry. The row selected by a token's id is its **static embedding**, written $\mathbf{e}(t)$. Training arranges the rows so that tokens used in similar ways sit near one another.

> COMMON MISREADING
>
> Individual embedding dimensions do **not** correspond to interpretable properties. Meaningful directions combine many coordinates, and the two-dimensional pictures everyone draws are projections that discard almost all of the structure.

Contextual representation 3.2 — written $\mathbf{h}_i^{(l)}$, the vector at position i after layer l, once attention has mixed in information

from other positions. One static embedding per token, many contextual representations: the static embedding depends only on the token, the contextual representation on the token *and its sentence*.

Attention 3.2, 3.3 — each position emits a query describing what it needs; every position offers a key describing what it has; each receives a weighted blend of the others' values. The scaling by $\sqrt{d_k}$ keeps the softmax out of its saturated region, where gradients vanish. Cost grows with the square of sequence length, so long contexts are computationally expensive.

> $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
>
> Attention weights show where information was gathered from. They are not an explanation of the output, and should not be presented as one.

Transformer 3.4 — stacked blocks, each pairing multi-head attention with a feed-forward network, with residual connections and normalisation around both. Attention moves information between positions; the feed-forward network processes information at a position. Residual connections are what make depth trainable.

Logits and softmax 3.5 — the forward pass ends with one raw score per vocabulary entry. Softmax exponentiates and normalises them into a probability distribution summing to one.

Temperature 4.3, 4.4 — divides the logits before the softmax. Below 1 sharpens the distribution, above 1 flattens it. Flattening promotes the *entire* vocabulary, and the tail is large: with ~200,000 tokens, a modest increase to each aggregates into most of the probability mass. **Top-p** truncates to the smallest set of tokens summing to p before sampling, which is why in practice it accompanies temperature rather than competing with it.

> $p(t) = e^{z_t/T} / \Sigma\, e^{z_{t'}/T}$
>
> Temperature adds no knowledge. It spreads the choice over more tokens, including unsuitable ones.

Autoregression 4.6 — one token per forward pass, appended to the context, then the whole thing runs again. Three consequences: cost grows with context length, early errors compound because they become input, and generated text becomes indistinguishable from supplied text.

Context window 4.7 — the maximum input in tokens, and zero-sum: instructions compete with material for the same space. A published length says what will be *accepted*, not how reliably material anywhere in it will be *used*. Retrieval of a specific detail becomes less reliable as input grows, and the middle of a long context is the weakest region.

Hallucination 4.8 — fluent output that is false. No step in the process checks a claim against the world; the model produces a plausible continuation, and plausible is not the same as true. This is the same mechanism that produces correct answers, operating where training data did not constrain it — which is why mitigations are architectural (retrieval, citation, verification) rather than corrective.

Pretraining and post-training 5.1 — pretraining on next-token prediction supplies the knowledge. **Supervised fine-tuning** then teaches the question-and-answer format, and **preference-based post-training** (RLHF and its relatives) installs tone, helpfulness and refusal behaviour — and with them a measurable tendency to agree with the person asking. Safety behaviour is trained disposition, not an enforced constraint.

THE SYSTEMS AROUND THE MODEL

Harness 5.4 — the program surrounding the model: it assembles the context, supplies the tool definitions, parses the output, executes tool calls, and decides when to stop. The same model behaves differently in different harnesses; much of what people attribute to the model is set by the harness. GitHub Copilot in VS Code is such a harness.

Tool and tool calling 5.8 — the model emits a structured request naming a tool and its arguments; the harness checks permissions and runs the code, then appends the result to the context. The model never executes anything. Tool descriptions are prompt text, and tool results are untrusted input.

Agentic loop 5.9 — observe, decide, act, repeat, until a stopping condition. Reliability depends heavily on the feedback available inside the loop: when the loop can check its result, for example by running tests or recalculating, it approaches the goal; without that feedback, errors tend to carry forward.

Retrieval (RAG) 5.11 — embed documents once, find those nearest to a question, place the retrieved passages in the context. The most effective single measure against hallucination, because it replaces recall with reading. Its failures are search failures, so debug it by inspecting what was retrieved, not by rephrasing the question.

Prompt injection 6.1, 6.2 — text placed where a model will read it, phrased as an instruction. **Indirect** injection arrives inside a document, web page or data record from outside. It works because instructions and data share one input channel and nothing in the architecture records provenance. It is a property of the architecture; no general fix exists at present.

What follows from how it works — summary of Part One

MECHANISM	IN PRACTICE
Parameters are learned from examples	Only a test with known answers shows whether a model can do a task
Training objective: predict the next token	Output is plausible, but not checked
Text is split into tokens	Counting letters and exact arithmetic are unreliable; cost and context are measured in tokens; other languages need more tokens than English
The next token is drawn	The same request can give different answers
Each generated token is appended	Long contexts cost more; early mistakes carry forward; nothing marks where text came from
Fixed parameters, limited context	No memory between conversations; recent and internal information is known only if it is in the context
No step checks statements	Hallucinations are to be expected; results must be checked
A harness surrounds the model	Much of what is attributed to "the AI" is a setting of the system

Part Two · What follows in practice

THE CONCEPTS IN GITHUB COPILOT

Where the concepts appear in GitHub Copilot in VS Code — as of October 2026, from secondary sources; names change between versions

CONCEPT	IN GITHUB COPILOT IN VS CODE
Model	A drop-down list in the chat. Which models appear is set by your organisation.
Context	Attached files and folders (with `#` in the chat box), the conversation so far, and, depending on settings, the open file.
Persistent instructions	Instruction files in the project, e.g. `.github/copilot-instructions.md`; prompt files (`*.prompt.md`) for recurring requests.
Working modes	*Ask*: questions and answers. *Plan*: a plan before execution. *Agent*: works on its own with tools.
Tools, agentic loop	In agent mode: reading and changing files, running terminal commands. Extensible through MCP servers.
Enforcement	Approval of tool calls and terminal commands; organisation policies.
Cost	GitHub AI Credits since 1 June 2026, charged by the tokens used; one credit is US$0.01. Code completion does not consume credits.

CHOOSING A MODEL, AND KNOWING IT WAS THE RIGHT ONE

Which characteristics matter for which work — rank these for the task in front of you, rather than looking for the best model

CHARACTERISTIC	MATTERS MOST WHERE
Depth on hard problems	Multi-step work in which an early error propagates through everything after it
Instruction following	Constraints that must hold across a long output, not just in the first paragraph
Structured output	A required shape — table, schema, fixed fields — reliably, every time
Tool use	Choosing the right tool, passing sound arguments, and reading the result correctly
Long-context handling	Material that sits far from the end of the window and still has to be used
Latency	Conversational back-and-forth, where waiting breaks the thread of thought
Cost	Anything run repeatedly, where a small per-call difference compounds

Reasoning effort 8.1 — a per-task setting on models that support it, not a fixed property of a model. Additional inference-time computation helps on tasks with genuine inferential depth; on retrieval or reformatting it adds cost and latency without improving the answer. How much it helps depends on the model, the task and the budget, so settle it by comparison on your own work rather than by assumption.

Benchmarks 8.2 — useful as a first filter, not as a decision. Composite indices hide a weighting someone else chose; test conditions vary between published figures; benchmark content leaks into later training data; saturated benchmarks compress real differences into noise; and none of them is your task. Use one to narrow thirty candidates to three, then stop.

Your own evaluation 8.3, 14.2 — five to ten real tasks with known-good outcomes; acceptance criteria written *before* the runs; each task run three times, because output is sampled and a single run measures luck; scored against the criteria with cost and elapsed time recorded alongside. An afternoon's work, and it answers the question a leaderboard cannot.

WHAT IT COSTS, AND WHAT THE COST ACTUALLY IS

Five units that are routinely confused — the conversion between them is not fixed

UNIT	WHAT IT IS	DEFINED BY
Token	A chunk of text processed	The tokeniser — technical, not commercial
Model call	One request with its own context	The system's architecture
Your instruction	One thing you asked for	You — and it may cause many calls
Credit	An accounting unit with a stated value	The vendor, by policy, and revisable
Allowance	Credits included per seat per period	Your organisation's plan

One instruction can trigger many model calls; an agentic workflow easily turns a single sentence into twenty. How tokens convert into credits is set by the vendor. 8.5

Token pricing 8.4 — input, cached input and output are priced separately. Output usually costs several times input, because generating it is sequential where reading is parallel. Cached input — a prefix the provider has already processed — is often around a tenth of input, which is the financial reason to keep standing instructions stable and put anything variable at the end. Reasoning tokens are billed as output and are frequently invisible in the interface.

What increases consumption. Long attached documents, re-sent on every turn of a conversation. Long conversations, because the entire transcript goes back each turn. High reasoning effort. Agentic loops, which add both steps and context at each step. Changing the early part of a prompt, which invalidates the cache. And retries caused by a task that was under-specified the first time.

> **THE COST THAT MATTERS**
> Model spend is usually the small term. Twenty minutes of verification against a loaded hourly rate commonly exceeds it several times over, so the cheapest route is the one that needs fewest corrections. Weigh model cost, elapsed time, retries, human verification, and the cost of an error that gets through. 8.7

CONTEXT ENGINEERING

What is in the context 9.1 — the vendor's system prompt, the tool definitions, your persistent instructions, material gathered from your working environment, retrieved passages, the conversation so far, and finally your words. Yours are a small fraction of the total. Many quality problems come from the context, often from parts of it that the interface does not show.

Long sessions. Harnesses manage growth differently: some drop the earliest turns, some summarise progressively, some retrieve selectively, and many re-insert standing instructions on every request. Which one yours does determines what quietly disappears. For work spanning sessions, end each with a short handover note and open the next with it. 9.4

Six elements of a prompt that works — the last two are most often omitted and change the result most

ELEMENT	WEAK	STRONG
Objective	"Compare the suppliers"	"A four-page memo recommending one supplier, for Thursday's decision meeting"
Background	*omitted*	"We run these continuously; sustained throughput matters more than peak"
Constraints	"Be accurate"	"Never present a vendor's claimed figure as a measurement"
Output shape	"Write it up"	"Comparison table, then conflicts, then assumptions, then recommendation"
Acceptance	*omitted*	"Every numeric claim traceable to a page number or row range"
Uncertainty	*omitted*	"List what you could not verify, and what you assumed instead"

WHEN TO START A NEW CONVERSATION

- The task has changed.
- Three corrections have failed — the context now holds three wrong approaches.
- Quality has drifted over a long session.
- You are moving from exploring to doing the actual work.
- Before anything where a stray earlier instruction would be expensive.

WORKING MODES AND AUTONOMY

Four modes — start at the least autonomous one that could work

MODE	YOU JUDGE	SUITS
Conversational	Every answer	Understanding, drafting, deciding what to do at all
Planning	The plan, before any work happens	Anything large, ambiguous or expensive to redo
Interactive	Each action, as it is proposed	Most substantial work, and any first run of anything
Autonomous	Only the final result	Established, checkable, repeated tasks

Choose on six questions: how ambiguous is the task; how reversible are the actions; what does a mistake cost; can success be checked automatically; what can the tools reach; how much undocumented judgement does it need. In GitHub Copilot, *Ask* corresponds to conversational and *Plan* to planning; *Agent* is interactive or autonomous depending on whether actions are approved one at a time or automatically. 10.2, 10.3

Three kinds of control, which are not interchangeable — §10.5

KIND	DETERMINES	EXAMPLE
Guidance prompts, instructions, style	What is *likely* to happen	"Cite a page number for every figure"
Orchestration steps, routing, stopping rules	What *ordinarily* happens	A review step that checks every citation before the draft is returned
Enforcement permissions, sandboxing, approval	What is *possible*	No write access outside the drafts folder

Writing "do not modify any files" is guidance, and guidance can be displaced by anything else in the context, including text that arrived in a document. Read-only access is enforcement and cannot. Put anything costly or irreversible in the third row. In GitHub Copilot an instruction file is guidance; approval of tool calls is enforcement.

PLANNING, DECOMPOSITION AND DELEGATION

When planning earns its cost. 11.1 The task is large, ambiguous, or expensive to redo; several people must agree what "done" means; the work will be delegated to someone or something else; or a first attempt has already surprised you. A planning pass costs a fraction of a full attempt and surfaces a misunderstanding in a two-minute artefact rather than in a finished deliverable.

A plan worth having 11.2 restates the objective in the model's own words, names the information it is missing, fixes the deliverable, lists constraints and acceptance criteria, decomposes the work, identifies dependencies, suggests a model and mode for each subtask, and says how the parts will be integrated. Reading the restated objective is the cheapest error-detection available: if it is wrong there, everything after it is wrong too.

When delegation is worthwhile. 11.4 The task exceeds what one context holds well; subtasks differ markedly in difficulty, so a cheaper model suits some of them; parts can genuinely run in

parallel; or the work recurs often enough to justify specifying it once.

What delegation costs. Context does not transfer between subtasks — each one starts without what the last one learned. Specification is real work. Errors propagate without being revisited. Integration is a judgement step routinely treated as clerical. And sequential dependencies add latency that parallelism does not recover. A single capable model with a well-curated context beats a poorly orchestrated pipeline more often than people expect.

PERSISTENT CONFIGURATION

Where an instruction belongs — push each one to the narrowest scope that still catches every case

SCOPE	COST PAID	GOOD FOR
One-off instruction	Once	Anything unusual about this particular request
Personal preferences	Every request, everywhere	Writing style, language, level of detail
Project instructions	Every request in that project	Terminology, conventions, standard layouts
Path-scoped instructions	Only on matching work	Rules that apply to one kind of material
Saved prompt template	When invoked	A request made repeatedly in the same form
Skill	Description always; body on demand	A procedure with standards attached
Organisational policy	Varies by platform	What must hold regardless of individual preference

The instruction hierarchy. 12.1 Providers generally train models to treat system-channel instructions as taking priority over user messages, and user messages over content inside documents. That ordering mostly holds. It is trained behaviour rather than an enforced rule, and conflicts *within* a level have no documented resolution at all — so remove contradictions rather than relying on one of them to win.

A style profile works when it names decisions 12.2, 12.4 — voice, tense, person, heading case, sentence length, words to avoid — rather than describing good writing in the abstract. "Write clearly" is too general to have an effect; "never open a paragraph with *Furthermore*" is a checkable rule. Review it quarterly and prune: every line is sent with every request, and instructions that no longer apply crowd out ones that do.

SKILLS, AGENTS AND TOOLS

Nine things that are regularly confused — §13.2

THING	SUPPLIES	ACTIVATED BY
Prompt	Text, once	You, typing it
Saved prompt file	Text, reused	You, by name
Persistent instructions	Text, always present	Nobody — always on
Skill	Text plus optional files, on demand	The model, where selection is automatic
Custom agent	Instructions, a tool set, sometimes a model	You, by selecting it
Subagent	The same, inside a larger workflow	The orchestrating system
Tool	The ability to act outside text	Model requests it; harness executes it
MCP server	A package of tools and resources	Configured once, then available
Hook	Code at a fixed point, every time	Nobody — it is deterministic

Skills and progressive disclosure. 13.2 A short name and description stay in context permanently; the body loads when the skill is judged relevant; supporting files open only if they are needed. Where a harness selects skills automatically, the description does unusual work — it is typically all the model has when deciding — so write it to say when the skill applies, not what it contains. Some environments require invoking a skill by name instead, which changes nothing about how to write the body.

ADDING AN MCP SERVER IS INSTALLING SOFTWARE
It is third-party code running with whatever credentials you give it. Establish the publisher, pin the version, read the tool descriptions once — they enter your context and are a prompt-injection surface — scope credentials to the minimum, and keep an inventory of what is installed. Enable only the tools the task needs. 13.4

How much structure to build — the last row is the one most often skipped

APPROACH	SETUP	APPROPRIATE WHEN
Single conversational request	None	One-off work, or you are still deciding what you want
Structured task specification	Minutes	It matters, and the first attempt missed
Saved prompt template	Under an hour	The same request recurs in the same form
Persistent instructions	An hour, plus upkeep	A preference spans many different tasks
Reusable skill	Half a day, plus upkeep	A multi-step procedure with standards, used regularly
Tool-supported agent	Setup and a permissions review	The work needs actions that text alone cannot perform
Several coordinated agents	Substantial	Subtasks differ markedly, or genuinely run in parallel
Conventional software	Often less than expected	**The procedure is exact and the inputs are structured**

Frequency, input variability, cost of error, number of people involved and how stable the task is all move the answer. A model earns its cost where inputs are irregular, judgement is required, or the task cannot be specified exactly in advance. Where none of those holds, a script is cheaper, faster and repeatable. 13.6

VERIFICATION

Write acceptance criteria before the work. 14.1 Written afterwards, they describe whatever was produced. Stated in the prompt, they improve the output as well as the review. The test of a criterion is whether someone other than its author could apply it and reach the same verdict.

DESIGN FOR CHEAP CHECKING

A document in which every figure carries its source is verified in minutes. The same document without sources takes an hour, because the checker must first work out what to check. Asking for sources therefore reduces the cost of checking considerably.

Seven dimensions, and a criterion you could actually apply — §14.1

DIMENSION	A CHECKABLE CRITERION
Factual accuracy	Each cited figure matches the page or row range cited for it
Completeness	All agreed criteria appear for all subjects, with gaps marked as gaps
Traceability	No claim without a source; “not stated” where a source is silent
Numerical correctness	Computed values reproduce when recalculated independently
Structural validity	Every required section is present and non-empty
Honest boundaries	Claims taken from a source are never presented as measurements
Repeatability	Three runs produce the same *figures*, if not the same prose

Proportionate oversight. 14.3 Exploratory work needs a skim. Material informing your own thinking needs spot checks on anything you will rely on. Work circulating to colleagues needs every factual claim traced. Anything entering a decision, a contract or an external communication needs full verification, with numbers recomputed independently.

ACTIONS DESERVING EXTRA CAUTION

- Anything irreversible: sending, filing, submitting, purchasing, deleting.
- Anything acting on material that came from outside the organisation.
- Anything with write access to a shared location.
- Numbers entering a decision, a contract or an external communication.
- Workflows combining private data, untrusted content and an outward channel — remove one of the three.
- First runs of anything, and first runs after a model or configuration change.

What not to paste. 14.4 Live credentials, personal data about identifiable people, material confidential to a third party, and

anything under legal hold. If a credential does reach a prompt, rotate it: a transcript is a logging surface. Your own organisation's ordinary working material is normally fine, which covers most of what people actually want to paste.

Where to read further

The course's annotated reading list, with forty-six entries sorted by what each one is good for, is at `../course/resources.html`. The five most useful starting points:

- **3Blue1Brown**, neural networks and transformers — a clear visual explanation of this material and a good place to start.
- **Andrej Karpathy**, *Deep Dive into LLMs like ChatGPT* for the full pipeline, and *Neural Networks: Zero to Hero* for building one from nothing.
- **Anthropic**, *Building effective agents* — design patterns, and a candid section on when not to use them.
- **Simon Willison**, prompt-injection archive — the running record of a problem that has not been solved.
- **OWASP Top 10 for LLM Applications** — a recognised vulnerability taxonomy, useful for talking to a security function in its own vocabulary.

Product documentation. For anything plan-dependent, version-dependent or priced, consult the vendor's own current documentation rather than this handout. GitHub Copilot, VS Code agent customisation, the Model Context Protocol and the Agent Skills specification each publish a reference that is updated more often than any course can be.

> VERIFICATION NOTE
> Product behaviour and all commercial figures in this handout were compiled in October 2026 from secondary sources, because the primary documentation hosts were not reachable from the environment in which it was written. The move to GitHub AI Credits on 1 June 2026 rests on search results about GitHub's announcement of April 2026. Confirm these details before standardising on any of them. Four questions only your own administrator can answer: which billing model your agreement is on; what the allowance is and whether it pools across the organisation; whether overage is enabled and what happens at exhaustion; and which report shows consumption, to whom.